

A U G R A D E

Fixed-point iteration as the system frame.

GNN is one mechanism choice inside that frame; the frame is what ships.

Malachi Mabie

Mabie Industries · for Krish, Augrade

Augrade is a fixed-point iteration problem with explicit, weak, discrete boundary conditions.



Mechanism layer

Closure, winding, family scoping, provenance. Deterministic by construction. The take-home submission ships this layer.



Learned residual

Relational consistency over typed graphs and calibrated edit proposals. GNN earns its keep here, where neighborhood context decides.



Validator boundary

Explicit, weak, discrete checks (closure, containment, clearance) kept outside the learned stack so plasticity cannot drift the geometry.

C o m p l e x i t y i s o b s e r v e r - r e l a t i v e

Once the shape is known, the problem stops being “complex” and becomes mechanism.

C O R E T H E S I S

Complexity is not intrinsic to the problem.

It is the mismatch between a problem and an observer.

The modeling task is to change the observer until the problem has shape.

Data complexity — measurable

graph size & density · entity entropy · rewrite variance · label noise · coverage residuals. *Use metrics, sweeps, and ablations.*

Learning complexity — observable

boundary conditions · representation target · validator loop · expert priors · active context. *Change the observer, then re-measure.*

W h e r e t h e v a l u e c o m p o u n d s

unknown soup

boundary conditions

latent geometry

generative reasoning

edit profit

There is shared geometry between neural representation and model behavior; the system should expose and train against that geometry.

The stack

Eight layers. The mechanism layer carries closure; the learned layer carries relations.

M8	Validator loop	deterministic, external
M7	Structured readout	typed edit proposals
M6	GNN consistency	relation propagation
M5	Prototype / memory	low-label regime
M4	Pair-relation scorer	sparse + conformal
M3	Graph construction	typed nodes + edges
M2	Object formation	supervectors
M1	Geometric perception	parser, snap, closure

Boundary

Learned residual

Mechanism layer

R e a d s a s

**Numerical computation
exposes the mechanism;**

*neural networks
learn the geometry.*

Lineage: Shi → Chern → fixed-point class theory → NMM/DDA. Deep Manifold names what we already built.

Supervector graph. Heterogeneous message passing. Deep-equilibrium friendly.

Nodes

primitive lines, arcs, polylines, circles, hatches, text, dimensions, inserts.

supervector wall faces, hatch boundaries, panel cells, candidate symbols, schedule tags.

face rooms, shafts, terraces — anything bounded.

annotation text, dimensions, leaders, colored marks. Layer name is provenance only.

Typed edges & mechanism

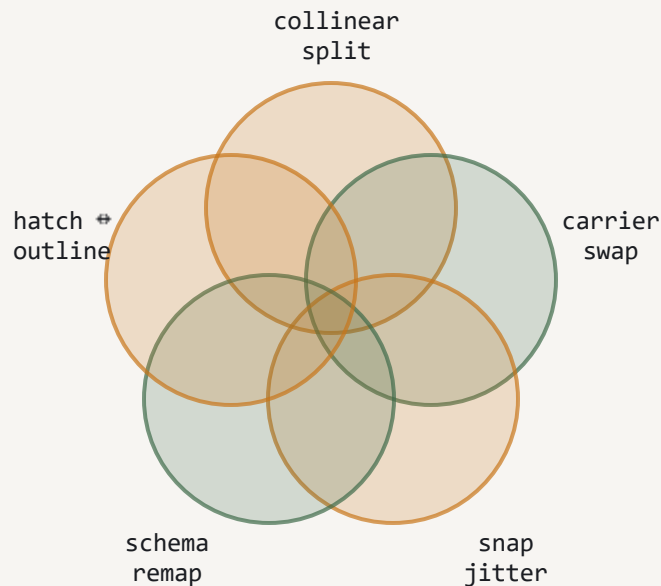
near, points_to, labels, inside, bounds, opens_into, same_symbol_family, same_schedule_tag, conflicts_with, parallel_offset, continuation, containment, duplicate_footprint.

Mechanism: heterogeneous message passing with relation-typed messages, R-rounds $\approx$ relation diameter (2–3 for floor plans). GraphSAGE for baseline; GATv2 for small-attention; PNA only when a relation justifies principal-neighborhood aggregation. Each round is a fixed-point iteration step.

Pool for geometry, tag for provenance. Lagrangian decomposition: $\|f(x) - x\|^2 + \lambda g(\theta)$. Pool collapses rewrite-equivalent inputs; tag preserves drafter intent as a side channel.

Rewrites: stationary equivalence classes

Five authored rewrites. Five coupled rings the scorer is asked to collapse.



M4 measures which rings have collapsed

Each rewrite is a categorical equivalence on authored convention, not a Euclidean group action.

Together they generate the stationary manifold the scorer must absorb. Score variance under M4 across rewrite kinds quantifies which sub-rings have collapsed and which haven't.

Operationalized as auxiliary loss: $L_{inv} = \mathbb{E}_g \|f(x) - f(g(x))\|^2$ during Phase 4.

C u r r i c u l u m — t h e s i m p l e p i p e l i n e

Five phases. Each phase is one DM boundary regime.

Phase	Task	Loss	Mechanism	Boundary regime
0	Graph export, deterministic	—	M1 + M2	<i>mechanism (no learning)</i>
1	Annotation alignment	—	M3	<i>mechanism</i>
2	Sparse pair-relation baseline	calibrated BCE + conformal head per relation	M3 sparse	<i>implicit / weak fixed-point</i>
3	Self-supervised contrast over rewrite orbits	InfoNCE on $(x, g(x))$ positives; random negatives	frozen encoder + head	<i>implicit / weak</i>
4	Small GNN supervised relations	$\text{BCE per relation} + L_{\text{inv}} = \mathbb{E}_n \ f(x) - f(g(x))\ ^2$	M5 (GraphSAGE/GATv2)	<i>semi-structured / intended</i>
5	Typed edit proposals under validator	validator-accepts $\wedge$ minimal-edit-cost (REINFORCE/GRPO)	M6 $\rightarrow$ M7	<i>explicit / weak / discrete (perturbed)</i>

I f w e w a n t t o d r e s s i t u p

Add components only when the simple pipeline shows a residual the new component is designed for.

M5 layer	Equivariant message passing (GATr-style multivector or ASEN-style permutation base + symmetry-breaking input)	<i>if rewrite orbits resist L_{inv}</i>
M5 head	Hopfield / dense associative memory over supervector embeddings (Kafraj-Krotov)	<i>carrier-style families recur</i>
M5 retrieval	Matryoshka graph embeddings — one representation, many product surfaces (32d cache · 256d routing · 512d offline)	<i>embedding size becomes a product knob</i>
Above M5	Sparse autoencoder bridge → tagged readout against codes/materials	<i>explainability becomes a deliverable</i>
M6 generative	Diffusion or autoregressive completion head for the inverse problem (plumbing inside a shell)	<i>Flavor 1 (system inference) becomes the priority</i>
M6 ranker	Bradley-Terry density-ratio head over frozen GNN embeddings (kalomaze pattern)	<i>edit acceptance is a 'know it when you see it' axis</i>

The structural reason this representation matches the editing problem.

LLM

State = KV cache.

Compressed integral path of past attention. Reusable while prefix is unchanged.

Context grows $O(n)$.

Sequential autoregressive prefix.

Updates are append-only.

Any token edit invalidates cache from that point onward.
Boundary-shift and fixed-point drift are unbounded by locality.

GNN over typed graph

State = per-node embedding cache.

Keyed by (sv_id, model_hash). Invalidate within L hops on graph mutation.

Context is the L-hop receptive field.

Not a token range. Bounded by graph locality, not by drawing size.

Updates are local.

Edit cascades = local re-message-passing. Fixed points are explicit (DEQ-GNN). Cheap to re-score during interactive use.

Local graph updates match edit propagation; foundation-model context handles global semantics; validators decide whether either one is allowed to act.

A linear head over a frozen GNN learns a density-ratio scorer for edit acceptance.

1

Freeze the GNN after Phase 4

Train a single $[d \rightarrow d]$ linear head on top with constant-sum BT logit. All discriminative capacity lives in W ; the aggregator is fixed. ~16M params over a much smaller backbone than 8B-class LMs.

2

Strip-and-flip primitive

Inject a typed-action / carrier-style hint into the supervector pair before encoding (e.g., 'treat this hatch as wall mass'). Strip the hint before the BT head sees the embedding. Flip chosen/rejected polarity by carrier class.

3

Match the chosen distribution

Maximize spread of accepted-edit scores rather than unbounded margin. Class-balance over (relation type, drafter convention, common-vs-rare). Calibrated scalar for M6, with conformal review bands kept intact.

Continual learning

The system improves week-over-week — without retraining.

THE LOOP



↻ Each iteration's residuals become the next iteration's labels — labels, rules, memory, weights all update.

WHY AEC ISN'T IID

Codes version

IBC, NEC, ADA, Title 24, IECC. State and local amendments multiply the surface. A 2018 baseline silently goes wrong.
→ **M7 validator update**

Best practices drift

Schedule conventions, tag carriers, hatch-vs-outline preferences evolve faster than full retrain cycles.
→ **rule mining back into the engine**

Project conventions vary

Each firm encodes legal and contractual intent in drafting standards. Same edit means different things across clients.
→ **per-project labels + prototype memory**

Code change → validator update, not model retrain. The learned residual only learns the new rare cases the validator surfaces.

Quality loops: make review worth doing

The human loop is not a tax. It is the product surface where expertise becomes data.

Operator sees

in the review surface

before / after overlay
model hypothesis
validator residual
similar past cases
one-click approve / reject / repair
why-it-matters explanation

System captures

into persistent memory

label / preference
correction trace
edit delta
validator before / after
time-to-fix
confidence shift
motif / rule candidate

Product improves

every loop iteration

fewer repeated errors
better retrieval
tighter conformal bands
stronger prototype memory
new deterministic rules
safer auto-apply threshold

Engagement is an ML systems variable: better DX creates better labels; better labels create better automation; better automation earns more trust.

verifiers + Environments Hub + hosted prime-rl. Useful for one specific thing, not the whole stack.

Where it fits

Phase 5 search-space coverage.

Wrap M7 validators (closure, containment, clearance, duplicate-footprint) as a verifiers Environment. Dataset = annotated drawing + corrupted variants. Reward = validator-accepts $\wedge$ minimal edit cost. Hosted prime-rl with LoRA for the typed-action policy.

Scale-out without infra cost.

Per-token pricing on a multi-tenant cluster, dedicated orchestrator per run. We run breadth experiments while the deterministic stack stays in our hands.

What we keep in-house

Mechanism layer (M1–M3). Closure, snap, supervector formation, graph export. The boundary-condition-camp commitment lives here.

Validators (M7). Stay deterministic. They become the verifier rubric for hosted RL — never the trained component.

Annotation loop. Calibrated review bands, expert feedback. Rule discovery flows back into the deterministic engine, not just into model weights.

When generic cognition gets cheap, the edge moves to boundary conditions, memory, and the compiler loop.

Commoditizes

what cheap intelligence eats

- code scaffolding
- paper search
- baseline model training
- synthetic rewrite generation
- eval harness boilerplate
- generic planning agents

Persists

boundary conditions only we can supply

- customer drawing history
- correction & rejection traces
- validator definitions
- false-merge policy
- code & jurisdiction constraints
- per-firm drafting conventions
- persistent object IDs & graph memory

Compounds

what gets stronger with use

- matryoshka graph embeddings
- prototype memory
- expert teaching models
- discovered supervectors & motifs
- rules compiled into engine
- verifier environments
- calibrated auto-apply policy

Borrow model capacity · own problem shape.

Twelve weeks to substrate. Twelve months to compounded learning.

F I R S T 1 2 W E E K S — s h i p t h e s u b s t r a t e

W1-2	W3-4	W5-6	W7-8	W9-10	W11-12
Graph export	Conversion	Sparse baseline	Rewrite tests	Small GNN	Demo bundle
<i>manifest, stable ids</i>	<i>DWG to graph + alignment</i>	<i>conformal bands</i>	<i>score stability</i>	<i>only if residual exists</i>	<i>decision gate</i>

M O N T H - 3 G A T E ≥ 1 relation baseline calibrated · rewrite report exists · GNN beats M3 on one context-heavy slice · validators still own truth.

M O N T H S 0 - 1 2 — t h e y e a r a r c

Q1 Months 0-3 Substrate M1 graph export · M2 annotation alignment · M3 calibrated baseline.	Q2 Months 4-6 Residual learning M4 rewrite invariance · M5 small GNN · prototype memory tests.	Q3 Months 7-9 Edit proposals M6 typed readout · validator deltas · workflow-exhaust labels.	Q4 Months 10-12 Product hardening interactive cache path · compute profiling · cross-project eval.
--	---	--	---

Branch by evidence · labels clean → GNN path · labels noisy → annotation/conversion first · residuals rich → strengthen validators · typed actions stable → hosted RL

A I + c o m p u t e — a c c e l e r a t e , t h e n p a c e

Spend compute where iteration is cheap. Slow down where semantics become product truth.

Accelerate hard

code scaffolds for exporters, parsers, manifests
batch feature inventories and ablation sweeps
synthetic rewrite generation at scale
dashboard / overlay QA screenshots
failure clustering and label-queue triage
short GPU bursts for GNN and memory heads

Pace intentionally

ontology: what counts as object, annotation, edit
validator definitions and acceptable residuals
false-merge thresholds and review-band policy
annotation semantics: schedule tag vs correction mark
deployment envelope: interactive CPU vs batch GPU
customer trust: which edits may auto-apply

Branch by evidence

baseline wins → do not force GNN
GNN wins context slice → add relation propagation
labels weak → invest in annotation alignment
rewrite variance high → fix quotient, not architecture
validators rich → verifier/search branch later
compute bottleneck → custom batched path, not earlier

Acceleration rule · spend compute on cheap falsification; spend human time on irreversible definitions.

O p e n q u e s t i o n s f o r t h e c a l l

What I want to learn from your end of the table.

- 1 Which authored rewrites are in scope for invariance: collinear split/merge, carrier swap, snap perturbation, schema remap, hatch-vs-outline carrier?
- 2 What does the annotation pipeline produce: primitive-level, supervector, component, relation, edit-event, or workflow trace?
- 3 Where is workflow exhaust captured today — revision history, validator residuals, designer comments, time-to-fix? Anything enumerable feeds Phase 1.
- 4 Deployment envelope: interactive CPU loop, batch GPU review, both? Decides PyG-flexible vs custom batched-tensor paths.
- 5 Which constraints are first-class: code/compliance, structural, MEP clearance, fire-rating, accessibility? Validator boundary should be enumerated, not hand-waved.
- 6 Appetite for hosted RL infrastructure (Prime Intellect Lab) for Phase 5 search, or all-in-house?

C l o s e

The smallest stack that survives contact with the product loop:

*reduce learning complexity, **learn stable latent geometries**, then **reason generatively over edit, state, and action space**.*

Not the smartest model — the environment in which smart models become useful, trusted, and compounding.

The frame ships; the geometry compounds.

The best discoveries compile back into the engine.